Secure Mobile Biometric Voting System

Project Report

Project Title: Secure Mobile Biometric Voting System
Application Name: Secure Voting
Tagline: Your Vote, Your Voice
Version: 1.0.0
Technology Stack: Flutter (Frontend) · Django REST Framework (Backend) · Python · Dart

---

Table of Contents

No	Title	Page
1	Introduction	10
2	Literature Survey / Existing System	18
3	Proposed System	23
4	System Design	27
5	Implementation	34
6	Source Code	39
7	Outputs	83
8	Future Enhancement	89
9	Conclusion	92
10	Bibliography	94

---

1. Introduction

1.1 Overview

The Secure Mobile Biometric Voting System is a comprehensive, full-stack digital voting platform engineered to modernize the process of democratic elections by making them accessible, secure,

transparent, and verifiable through mobile technology. In today's increasingly digital world, the demand for reliable and efficient electronic governance solutions has never been greater. Voting, as the cornerstone of democracy, deserves the highest standards of technological reliability, and this system is built with that mission at its core.

Traditional paper-based voting systems suffer from several inherent limitations — they are time-consuming, require extensive physical infrastructure, are susceptible to human error and fraud, and often result in delays in counting and announcing results. Even partially computerized systems frequently rely on outdated methods that fail to address the fundamental problem of voter identity verification. The Secure Mobile Biometric Voting System addresses all of these issues head-on by combining modern mobile app development, a robust RESTful backend API, cryptographic vote hashing, token-based authentication, and automated email communication into a single unified platform.

The application is named "Secure Voting" and carries the tagline "Your Vote, Your Voice" — reflecting a philosophy that centers the voter as the most important stakeholder in any election. The system is designed so that every eligible citizen can register, receive a digital voter identity, participate in elections from their mobile device, and verify that their vote has been recorded — all without setting foot in a physical polling station.

1.2 Motivation

The primary motivation behind this project is the widespread issue of electoral fraud and disenfranchisement. In many parts of the world, voters are turned away due to administrative errors on voter rolls, impersonation of voters is a documented problem, and the logistics of setting up physical polling stations in remote or inaccessible areas often prevents significant portions of the population from participating. Digital voting, when implemented securely, eliminates each of these barriers.

Additionally, the rapid proliferation of smartphones — even in developing economies — makes mobile-based solutions highly practical. A mobile voting application can reach voters anywhere with an internet connection, extending the democratic process far beyond the boundaries of traditional voting infrastructure.

The biometric authentication aspect of this system is particularly motivated by the growing integration of fingerprint sensors and face recognition in everyday consumer smartphones. By leveraging the device's own trusted execution environment (TEE) for biometric verification, the system avoids the cost and complexity of dedicated biometric hardware while still providing a high level of identity assurance. The device performs the fingerprint match, issues a biometric token, and the server validates that token — creating a chain of trust without transmitting any actual biometric data over the network.

1.3 Objectives

The primary objectives of this project are as follows. First, to provide a secure and authenticated voter registration portal where citizens can apply to be registered voters, uploading their photographs and personal details through a guided form. Second, to implement a robust administrator panel that allows election officials to manage the entire voter registration lifecycle — reviewing applications, approving, rejecting, or blocking voters — and to manage elections, parties, candidates, and results from a single dashboard. Third, to enable secure electronic voting where each approved voter can cast exactly one vote per election, with the vote being recorded along with a SHA-256 cryptographic hash that serves as proof of participation. Fourth, to support biometric login as an alternative authentication method, making login faster and more secure for regular users. Fifth, to automate all critical communications through an email notification system, ensuring voters are always informed of their application status, upcoming elections, and result publications. Sixth, to provide voters with a digital voter card that mirrors the format of a traditional government-issued identity card, making the transition to digital voting more familiar and intuitive. Seventh, to allow voters to raise correction requests for any inaccuracies in their registered details, with a full admin review workflow.

1.4 Scope

The scope of this system encompasses the entire lifecycle of a digital election — from voter onboarding and identity verification, through election creation and management, to voting and result publication. It is designed for deployment by election commissions, institutional bodies, student government organizations, cooperative societies, residential associations, or any organization that requires a secure, auditable voting mechanism. The system is built with scalability in mind, capable of handling multiple simultaneous elections, thousands of voters, and complete concurrent data management.

The system is platform-independent on the frontend — the Flutter framework allows the same codebase to compile and run on Android, iOS, macOS, Windows, Linux, and Web — making it deployable across virtually any device ecosystem. The backend is equally portable, running on any server that supports Python and Django, from small virtual private servers to enterprise-grade cloud infrastructure.

---

2. Literature Survey / Existing System

2.1 Traditional Paper-Based Voting

Paper-based voting has been the dominant form of electoral participation for centuries. In this system, voters physically present themselves at designated polling stations, verify their identity against a printed voter roll, receive a ballot paper, mark their choice, and deposit it in a sealed ballot box. While this method has the advantage of being universally understandable and tangible, it is plagued by a wide range of vulnerabilities. Ballot stuffing, impersonation of absent voters, destruction of ballot boxes, manipulation during counting, and delays caused by geography and logistics are all well-documented

problems. The human element in the counting process also introduces the risk of honest mistakes that can alter outcomes in closely contested elections.

2.2 Electronic Voting Machines (EVMs)

Electronic Voting Machines represent a significant improvement over paper ballots in terms of counting speed and accuracy. Countries like India have deployed EVMs at scale, with millions of votes cast across thousands of booths in a single election day. EVMs record votes electronically, eliminating counting errors and dramatically speeding up result declaration. However, EVMs still require physical presence at a polling station, require costly hardware procurement and maintenance, and have faced scrutiny over their vulnerability to tampering. The standalone nature of most EVM deployments means they do not benefit from network-based real-time monitoring, and any software compromise may go undetected until after results are declared.

2.3 Internet-Based Voting Systems

Several countries have experimented with internet-based voting for certain elections. Estonia is the most prominent example, offering i-voting since 2005 using national digital identity cards. The Estonian system has demonstrated the practical viability of internet voting at a national scale, with millions of votes cast online over multiple election cycles. However, academic cryptographers and security researchers have consistently raised concerns about the Estonian system, particularly around the difficulty of ensuring voter secrecy, protecting against client-side malware on the voter's device, and the concentration of trust in a few centralized servers.

Other internet voting experiments, such as those attempted in France, Switzerland, and certain jurisdictions in the United States, have faced similar criticism and have often been suspended due to identified security vulnerabilities. These experiences have highlighted that internet voting requires extremely careful design, open-source auditability, and layered security mechanisms to be trustworthy.

2.4 Mobile Voting Research

Academic literature on mobile voting emphasizes that smartphone-based voting systems must address three fundamental requirements simultaneously: authentication (ensuring the voter is who they claim to be), privacy (ensuring the voter's choice cannot be linked back to their identity), and integrity (ensuring votes cannot be altered or fabricated after submission). Research papers such as "Civitas: Toward a Secure Voting System" (Clarkson et al., 2008) and "Helios: Web-based Open-Audit Voting" (Adida, 2008) have explored cryptographic approaches to achieving all three properties simultaneously.

The integration of biometric authentication in mobile voting has been studied as a solution to the authentication problem. Device-level biometrics — where the smartphone's trusted execution environment handles fingerprint matching without exposing raw biometric data to the network — are

considered more privacy-preserving than server-side biometric databases, since no biometric data leaves the user's device.

2.5 Gaps in Existing Systems

Through analysis of the above systems, several critical gaps emerge that the proposed Secure Mobile Biometric Voting System aims to address. Existing systems typically lack a unified, user-friendly mobile interface that combines voter registration, identity management, election participation, and result viewing in a single application. They rarely include automated email communication that keeps voters informed throughout the process without any manual effort from administrators. Digital voter card functionality — which provides voters with a familiar, ID-card-format digital document — is absent from most systems. Voter correction workflows, allowing voters to request corrections to their registered details and have those corrections reviewed by an admin, are similarly absent. Finally, real-time election monitoring dashboards that give election officials live vote counts and participation data during an ongoing election are not commonly available in freely deployable systems. The proposed system addresses all of these gaps.

---

3. Proposed System

3.1 System Overview

The proposed Secure Mobile Biometric Voting System is a mobile-first digital voting platform that provides an end-to-end solution for electoral management. The system is built on a client-server architecture: a Flutter mobile application serves as the client interface for both voters and administrators, while a Django REST Framework backend provides the API layer, business logic, and data persistence. Communication between the client and server occurs exclusively via HTTP REST API calls, with all authenticated requests carrying a bearer token issued at login.

The system supports two distinct user roles. The Voter role encompasses all registered citizens who have been approved to participate in elections. The Admin role is held by election officials who have full control over the system. A single application handles both roles, with a login screen toggle allowing users to select whether they are logging in as a voter or as an administrator.

3.2 Voter Registration and Identity Management

Voter registration is the entry point into the system for all citizens. A prospective voter can open the application and navigate to the registration screen by tapping "Apply for Voter" on the login screen. The registration form collects the voter's full name, father or husband's name, date of birth, gender, complete residential address, email address, mobile number, and a photograph. The photograph is uploaded using the device's camera or gallery picker, supporting both Android and iOS file access. A

passcode chosen by the voter during registration is hashed using Django's PBKDF2 algorithm before being stored, ensuring that even if the database is compromised, raw passcodes cannot be recovered.

Upon successful submission, the registration is placed in a "pending" state and a confirmation email is automatically sent to the voter's email address, acknowledging receipt of the application and setting expectations about the review timeline. The admin then reviews the application and either approves or rejects it. Upon approval, the system generates a unique Voter ID using the prefix "VAL" followed by six randomly generated digits, with a loop ensuring no duplicate voter IDs exist. This Voter ID, along with login instructions, is sent to the voter via an HTML-formatted email. The voter can then log in using their new Voter ID and the passcode they set during registration.

3.3 Authentication and Security

The system implements three authentication modes. Standard authentication uses a Voter ID and passcode combination, where the submitted passcode is checked against the stored hash using Django's `check_password` utility. Biometric authentication allows voters who have enabled it to log in using their smartphone's fingerprint sensor. The device performs the actual biometric match in its trusted hardware, then provides a biometric token to the application, which submits it to the server along with the voter ID. The server validates the token against the one stored when the voter enabled biometric login. Admin authentication uses a Django username and password combination, verified through Django's built-in authentication system, with an additional check that the user has staff privileges.

All authenticated sessions are managed using DRF's Token Authentication. Upon successful login, the server issues a unique token for the session. This token is stored locally on the device using SharedPreferences and included in the Authorization header of every subsequent API request. Logging out deletes the token from both the server and the local storage, effectively invalidating the session.

3.4 Election Lifecycle Management

Elections in the system move through a well-defined lifecycle: Upcoming → Live → Closed → Results Published. Admins create elections by specifying a name, description, start date, and end date. An election begins in the "upcoming" state and can be started manually by the admin, transitioning it to "live." When an election goes live, the system automatically sends an email notification to every approved voter who has provided an email address, informing them of the election name, description, and voting window.

While an election is live, admins can monitor it in real-time through the monitoring endpoint, which returns current vote counts per candidate, total votes cast, and participation percentage. Admins can also extend the election's end date if needed. When the admin decides to close the election, its status changes to "closed" and no further votes can be cast. Admins can then access the full results — ranked

candidates with vote counts and percentages, the winner, and overall participation rate — and when satisfied, publish the results, making them visible to all voters through the app.

3.5 Vote Casting and Integrity

The vote casting mechanism is designed to enforce two critical constraints: only approved voters can vote, and each voter can cast exactly one vote per election. When a voter taps "Cast Your Vote" on a live election, the application loads the candidate list for that election from the server. Each candidate is displayed with their name, party name, and party symbol image. The voter taps on a candidate to select them, after which a summary confirmation dialog appears showing the selected candidate's details and a warning that the vote cannot be changed once submitted.

Upon confirmation, the vote is sent to the server's cast-vote endpoint. The server performs a series of validations: it confirms the voter's account is approved, confirms the election is currently live, and most importantly, checks whether a vote record already exists for this voter and election combination — enforced at the database level using a unique-together constraint. If all validations pass, a SHA-256 hash is computed from the voter ID, election ID, candidate ID, and current timestamp concatenated together, producing a unique 64-character hexadecimal string. This hash is stored with the vote record and returned to the voter's application, where it is displayed in the success dialog as a "Transaction Hash" that the voter can copy and retain as proof of their participation.

3.6 Voter Card and Corrections

Each approved voter can view their digital voter card within the application. The voter card screen renders the voter's registered information — photo, full name, voter ID, father's name, date of birth, gender, and address — in a card layout styled to resemble a formal government-issued identity document. This digital card provides voters with immediate access to their registration details without needing to visit any government office.

If a voter identifies an error in their registered name or father's name, they can submit a correction request through the app. The correction request captures the desired corrected values for both fields. Admins review pending correction requests through the correction management screen. Upon approval, the system automatically updates the voter's record with the corrected details and sends an email notification confirming the change. Upon rejection, the voter receives an email explaining the outcome, with any notes the admin chose to include.

3.7 Email Notification System

Automated email communication is deeply integrated into every major workflow in the system. The email service is implemented as a dedicated Python module that sends HTML-formatted emails using Django's `EmailMultiAlternatives` class, which sends both a plain-text version and a rich HTML version for compatibility with all email clients. Every email sent by the system is logged in the

`EmailNotification` database table, recording the recipient's email and name, the notification type, subject, body, whether the send was successful, and any error message in case of failure. This audit log allows admins to identify voters who may not have received critical communications and take remedial action.

The notification types include registration confirmation (sent immediately after a voter submits their application), voter approval (sent with the voter's ID and login instructions), voter rejection (sent with reason if provided), new election announcement (sent to all approved voters when an election goes live), correction request approved (sent with the updated details), and correction request rejected (sent with admin notes). Each email is visually styled with a consistent brand identity — dark blue header with the system name in yellow, structured content sections, and a footer with copyright information.

---

4. System Design

4.1 Architecture

The Secure Mobile Biometric Voting System follows a three-tier architecture comprising a presentation tier (Flutter mobile app), a logic tier (Django REST Framework API), and a data tier (SQLite/PostgreSQL database). The presentation tier handles all user interactions and renders data received from the API. The logic tier validates all inputs, enforces business rules, manages authentication, performs cryptographic operations, and coordinates all data access. The data tier persistently stores all application data including voter records, election data, vote records, and email logs.

The Flutter app communicates with the Django backend exclusively through HTTP REST API calls. All requests to protected endpoints include a `Token <value>` header in the Authorization field. The server validates this token against its database for every request. Media files — voter photos, candidate photos, and party symbols — are served as static files from the backend server and referenced by their relative URL path in the API response. The Flutter app constructs the full URL by prepending the media base URL constant to the relative path returned by the server.

4.2 Database Schema

The database layer consists of seven interconnected models, each representing a core entity in the voting domain.

The **Voter** model is the central entity for all registered citizens. It stores a one-to-one link to Django's built-in User model (created lazily when the voter first logs in), a unique voter_id string, full_name, father_name, date_of_birth, gender (M/F/O), address, email, mobile_number (unique across all voters), a photo image file, the hashed passcode, a status field with four possible values (pending, approved,

rejected, blocked), a boolean biometric_enabled flag, a biometric_token string for device-level biometric binding, and automatic created_at and updated_at timestamps.

The **Election** model stores each election's name, description, start_date, end_date, status (upcoming, live, closed), a reference to the admin user who created it, a total_votes counter, a result_published boolean flag, and automatic timestamps. The model also exposes an `is_active` computed property that returns True only when the status is live and the current time falls between the start and end dates.

The **Party** model is a simple entity storing the party name, an optional symbol image, a text description, and a creation timestamp. Political parties are independent of specific elections, allowing the same party to participate in multiple elections through different candidate registrations.

The **Candidate** model links a specific person to a specific election and party, storing their name, an optional photo, a bio description, and a votes_count integer that is incremented atomically each time a vote is cast for that candidate. The unique-together constraint on (election, party) enforces that each party can have at most one candidate per election.

The **Vote** model is the most critical record in the system. It stores a foreign key to the voter, election, and candidate, an auto-set voted_at timestamp, and the 64-character SHA-256 vote_hash. The unique-together constraint on (voter, election) is the primary enforcement mechanism preventing double voting. The vote_hash field has a unique constraint at the database level as well, preventing hash collisions from creating duplicate records.

The **VoterCorrection** model tracks each correction request submitted by a voter, storing references to the voter, the requested new full_name and father_name values, the current status (pending, approved, rejected), any admin notes, and timestamps.

The **EmailNotification** model serves as a complete audit log of all outgoing emails. It records the recipient_email, recipient_name, notification_type (from a defined set of choices), subject, body, sent_at timestamp, a success boolean, and an optional error_message field for failed sends.

4.3 API Design

The API is organized into logical groups using Django REST Framework's DefaultRouter for CRUD ViewSets and manual URL path declarations for custom action endpoints. The router automatically generates list, create, retrieve, update, partial-update, and destroy endpoints for five registered ViewSets: voters, elections, parties, candidates, and corrections.

Custom action endpoints are registered using DRF's `@action` decorator on the respective ViewSets. The VoterManagementViewSet exposes approve, reject, block, unblock, and remove_duplicates actions. The ElectionViewSet exposes start, stop, extend, results, publish_results, and monitoring actions. The VoterCorrectionViewSet exposes approve and reject actions.

Standalone function-based views handle authentication flows (admin login, voter login, biometric login, voter registration, enable biometric, logout), the admin dashboard statistics endpoint, vote casting, vote status checking, voter profile retrieval, voter election listing, and voter-facing result viewing. All endpoints return JSON responses. Error responses always include an `error` key with a descriptive message.

4.4 Authentication and Authorization

The system uses DRF's built-in TokenAuthentication globally. Each user (both admin users backed by Django's User model and voter users created dynamically at first login) has an associated token in the authtoken_token table. This token is created on login and deleted on logout.

Permission classes enforce role-based access control at the view level. Admin-only endpoints use the `IsAdminUser` permission class, which checks the `is_staff` flag on the associated User. Voter-accessible endpoints use the `IsAuthenticated` permission class. A few public endpoints — health check, admin login, voter login, biometric login, and voter registration — use `AllowAny`, since they must be accessible without a pre-existing session.

4.5 Frontend Architecture

The Flutter frontend is organized using the Provider state management pattern. A single `AuthProvider` ChangeNotifier class manages the global authentication state, including the current user's token, role (admin or voter), voter data, and any error or loading state. The `AuthProvider` delegates all API calls to the `ApiService` singleton and persists state across app restarts using `SharedPreferences`.

The `ApiService` class implements the singleton pattern, ensuring a single HTTP client instance is reused throughout the application's lifecycle. It exposes typed methods for every API endpoint, handling JSON encoding, multipart file uploads for images, authorization header injection, response parsing, and error mapping. Network errors are surfaced as `ApiException` objects containing the HTTP status code and a human-readable message, which the UI layer catches and displays as snackbars or inline error messages.

The theme system is centralized in `AppTheme`, defining a consistent visual language: a primary yellow color (`#FFD600`), a dark accent color for voter actions, a surface color for card backgrounds, typography using Google's Inter font family (loaded via the `google_fonts` package), standardized card shadows at two intensity levels (soft and standard), and gradient definitions for all button and header styles.

4.6 Serializers and Data Validation

Django REST Framework serializers handle all data validation and transformation between HTTP request/response bodies and Python model instances. The `VoterRegistrationSerializer` processes voter self-registration, applying passcode hashing in its `create()` override so that the raw passcode never reaches the database. The `VoterSerializer` provides a read-focused view of voter data for admin management screens, exposing all personal details except the passcode. The `ElectionSerializer` includes computed fields — `candidates_count` and `total_votes_cast` — calculated using SerializerMethodField to avoid redundant queries. The `CandidateResultSerializer` similarly computes a `vote_percentage` by dividing the candidate's votes_count by the election's total vote count, rounded to two decimal places. The `DashboardSerializer` is a non-model serializer that structures the flat dictionary returned by the dashboard view into a validated, typed response.

---

5. Implementation

5.1 Backend Implementation

The backend is implemented in Python 3 using Django 4.x and Django REST Framework 3.x. The project is structured as a standard Django project with a main project package (`voting_backend`) and a single API application (`api`). All domain logic — models, views, serializers, URLs, email service, admin registration, and signals — lives within the `api` app, keeping the project organized and easy to navigate.

The database used during development is SQLite, which requires no additional installation and is appropriate for a single-server deployment. The database file is created automatically by Django's migration system. The migration history for the `api` app is stored across five migration files: the initial migration creating Voter, Election, Party, Candidate, Vote, and their relationships; a migration to alter the voter_id field to allow null values (enabling pre-approval registration); a migration adding the father_name field to Voter; a migration creating the VoterCorrection model; and a migration adding the EmailNotification model and the email field to Voter.

Django's signals system is used via `api/signals.py` to hook into model lifecycle events, enabling automated responses to state changes without cluttering the view code. The `AppConfig` class in `api/apps.py` connects these signals when the application starts.

Media file handling is configured in the Django settings to store uploaded files in a `media/` directory organized into subdirectories: `voter_photos/` for voter registration photos, `candidate_photos/` for candidate profile photos, and `party_symbols/` for party logo images. The `MEDIA_URL` and `MEDIA_ROOT` settings expose these files through the development server. In production, a dedicated file server or CDN would serve these media files.

5.2 Voter ID Generation Logic

When a voter application is approved, the system generates a unique Voter ID using a specific algorithm. A three-character prefix "VAL" (standing for Voter Application List or Voter ALlocation) is combined with six random decimal digits, producing IDs in the format `VAL123456`. A uniqueness check loop queries the database for any existing voter with the generated ID; if a collision is found, new random digits are generated until a unique ID is obtained. The probability of collision is extremely low for small voter populations and becomes practically negligible given that the loop will always resolve to a unique value.

The same generation logic is applied in two places: when an admin directly creates a voter (auto-approved) and when an admin approves a pending self-registration. This ensures consistent ID format regardless of the registration pathway.

5.3 Vote Hash Generation

The cryptographic vote hash is generated using Python's built-in `hashlib` library with the SHA-256 algorithm. The input to the hash function is a string formed by concatenating the voter's voter_id, the election's database ID, the candidate's database ID, and the ISO 8601 formatted current timestamp, joined by hyphens. This string is encoded to bytes using UTF-8 encoding and passed to `hashlib.sha256()`. The resulting 32-byte digest is converted to a 64-character lowercase hexadecimal string using the `.hexdigest()` method.

The inclusion of the timestamp in the hash input means that even if two voters cast votes for the same candidate in the same election at nearly the same time, their hashes will differ. The unique constraint on the `vote_hash` field at the database level provides an additional safety net. The hash is returned to the voter's mobile app upon successful vote submission and displayed on screen as the "Transaction Hash," which the voter can note down as proof of their vote.

5.4 Email Service Implementation

The email service module (`api/email_service.py`) implements five public functions, each responsible for a specific type of notification. Each function constructs both a plain-text and an HTML version of the email body. The HTML versions use inline CSS styling — appropriate for email clients that strip external stylesheets — featuring a gradient header, structured content cards, and a consistent footer. The `EmailMultiAlternatives` class from Django's mail module is used to send both content versions in a single MIME email, allowing email clients that support HTML to render the rich version while those that do not fall back to plain text.

After each send attempt — successful or not — the email is logged to the `EmailNotification` model through the private `_log_email()` helper function. This function creates a database record

with all email details and the outcome. If an exception occurs during sending, the exception message is captured and stored in the `error_message` field of the log record, and the `success` field is set to False. This design ensures that the application never silently loses track of communication failures.

The `send_new_election_email()` function iterates over all approved voters with non-empty email addresses and sends each one an individualized email. It returns a dictionary with counts of successful and failed sends, which the election-start API endpoint includes in its response, allowing the admin to know if any notifications failed.

5.5 Flutter App Implementation

The Flutter application is structured into five main directories under `lib/`. The `main.dart` file at the root of `lib/` serves as the application entry point and hosts the splash screen widget. The `providers/` directory contains `auth_provider.dart`, the global state manager. The `services/` directory contains `api_service.dart`, the HTTP client layer. The `utils/` directory contains `constants.dart` for app-wide string constants and `theme.dart` for visual theme definitions. The `screens/` directory is subdivided into `auth/` (login and registration screens), `admin/` (all admin-facing screens), and `voter/` (all voter-facing screens).

The splash screen uses a dual animation — an elastic scale-up animation from 0.5x to 1.0x scale, and a simultaneous fade-in from 0 to 1 opacity — to create an engaging launch experience. After a two-second delay, the screen checks the persisted authentication state and navigates to either the Admin Dashboard, the Voter Home Screen, or the Login Screen as appropriate, using a fade transition.

5.6 State Management with Provider

The `AuthProvider` class extends Flutter's `ChangeNotifier` and serves as the single source of truth for authentication state throughout the application. It exposes the following observable properties: `isLoggedIn` (bool), `role` (String — either "admin" or "voter"), `voterData` (Map), `isLoading` (bool), and `error` (String?). Any widget in the tree can subscribe to changes using `Consumer<AuthProvider>` or `context.watch<AuthProvider>()`.

The `init()` method is called during splash screen initialization to restore session state from SharedPreferences. The `adminLogin()` and `voterLogin()` methods call the respective API service methods, save the returned token and role using SharedPreferences, and notify all listeners of the state change. The `logout()` method clears all persisted state and notifies listeners, causing the widget tree to rebuild and navigate to the login screen.

5.7 Admin Dashboard Screen

The Admin Dashboard is the central hub for administrators and is the first screen they see upon successful login. The screen is built as a stateful widget that calls the `/api/admin/dashboard/` endpoint on initialization and whenever the user pulls down to refresh. The dashboard displays ten statistics organized in a two-column grid: Total Voters, Approved Voters, Pending Approvals, Rejected Voters, Blocked Voters, Total Elections, Live Elections, Upcoming Elections, Closed Elections, and Total Votes Cast. Each statistic card displays a relevant icon, a label, and the current count.

Below the statistics grid, the dashboard displays five navigation tile cards: Voter Management, Election Management, Party Management, Candidate Management, and Results. Each tile navigates to its respective management screen. A sixth tile navigates to the Correction Management screen. The top header section uses the primary yellow background color with the app logo and a logout button.

5.8 Voter Management Screen

The Voter Management screen provides admins with a filterable, searchable list of all registered voters. The screen loads voters from the API with optional status filtering (All, Pending, Approved, Rejected, Blocked) and a text search that queries against the voter's name, voter ID, or mobile number. The voter list is paginated to prevent excessive data loading.

Each voter entry in the list displays the voter's name, voter ID, mobile number, registration date, and current status with a color-coded badge. Tapping a voter expands an action panel showing buttons appropriate to the voter's current state — a pending voter shows Approve and Reject buttons, an approved voter shows a Block button, and a blocked voter shows an Unblock button. Approving a voter triggers an API call that assigns a voter ID and sends an approval email; this outcome is reflected immediately in the UI.

5.9 Election Management Screen

The Election Management screen lists all elections sorted by creation date. Each election card displays the name, status badge, start and end dates, number of candidates, and total votes cast. Admins can create new elections through a dialog form that captures the name, description, start date, and end date using date and time pickers. Tapping on an existing election reveals management options: Start Election (if upcoming), Stop Election (if live), Extend Election (if live or closed), View Results, Publish Results (if closed), and Real-Time Monitoring (if live). The monitoring option opens a live view of vote distribution among candidates.

5.10 Party and Candidate Management

The Party Management screen allows admins to create and delete political parties. Creating a party requires a name, optional description, and an optional symbol image uploaded from the device's files. Party symbols are displayed throughout the app wherever party information appears — on candidate cards during voting, in result screens, and in the admin candidate list.

The Candidate Management screen allows admins to add candidates to specific elections. Each candidate is linked to an election and a party (with the constraint that a party can have only one candidate per election). Admins provide the candidate's name, an optional photo, an optional biography, and select the election and party from dropdown menus populated from the server. Candidates can also be edited and deleted, with the backend cleaning up associated photo files upon deletion.

5.11 Vote Casting Screen

The Vote Casting screen is one of the most carefully designed screens in the application from a UX perspective. It is accessed by tapping "Cast Your Vote" on a live election card in the Voter Home Screen. The screen displays a dark gradient header with the election name and a progress step indicator showing two steps: Select and Submit.

The candidate list is rendered as a scrollable collection of animated cards. Each card shows the party symbol, candidate name, and party name. When a voter taps a candidate, the card animates with a highlight border and a check-mark radio indicator, and the card's shadow shifts to reflect the selection. The "Submit Vote" button appears at the bottom only after a candidate is selected, preventing accidental navigation.

Tapping Submit Vote triggers a confirmation dialog that summarizes the selected candidate and party, and prominently warns that the vote cannot be changed once submitted. The confirmation dialog's "Vote Now" button sends the vote to the server. Upon success, a full-screen success dialog displays the transaction hash in a selectable monospace font (Roboto Mono), allowing the voter to copy their proof of participation. Tapping "Back to Home" returns to the Voter Home Screen, which refreshes to show the "Vote Recorded" status on the election card.

---

6. Source Code

6.1 Backend Source Files

`voting_backend/voting_backend/settings.py` — The Django project configuration file. It defines INSTALLED_APPS (including `rest_framework`, `rest_framework.authtoken`, `corsheaders`, and `api`), the database configuration (SQLite for development), the MEDIA_URL and MEDIA_ROOT for file uploads, DEFAULT_AUTO_FIELD set to BigAutoField, the authentication backends, REST_FRAMEWORK settings (default authentication class: TokenAuthentication, default permission class: IsAuthenticated), and the email backend configuration (SMTP settings for the notification service). The DEFAULT_FROM_EMAIL is configured with the election commission's address.

`voting_backend/voting_backend/urls.py` — The root URL configuration. It includes the `api/` prefix routing to the API application's URL patterns and configures the media file serving for

development.

`voting_backend/api/models.py` — Defines all seven database models: Voter, Election, Party, Candidate, Vote, VoterCorrection, and EmailNotification. Key design decisions include: the Voter model's `status` field using string choices for human-readable values; the Vote model's `save()` override generating the SHA-256 hash if not already set; the Election model's `is_active` property computing live status from both the status field and the current datetime; and all models including ordering in their Meta classes for consistent default list ordering.

`voting_backend/api/serializers.py` — Implements thirteen serializer classes. `VoterRegistrationSerializer` overrides `create()` to hash the passcode using `make_password`. `VoterLoginSerializer`, `BiometricLoginSerializer`, and `AdminLoginSerializer` are non-model serializers handling authentication inputs. `ElectionSerializer` adds computed `candidates_count` and `total_votes_cast` fields via `SerializerMethodField`. `CandidateSerializer` uses `source` to include the party name and symbol from the related Party model. `CandidateResultSerializer` adds a `vote_percentage` computed field. `DashboardSerializer` is a flat non-model serializer for the ten dashboard statistics. `ElectionResultSerializer` composes other serializers to structure complete result data. `VoterCorrectionSerializer` exposes read-only voter_name and voter_id from the related Voter.

`voting_backend/api/views.py` — The largest file, implementing all API business logic across 770 lines. It is organized into six sections: Authentication Views (admin_login, voter_login, biometric_login, voter_register, enable_biometric, logout_view), Admin Dashboard (admin_dashboard), Voter Management (VoterManagementViewSet with approve, reject, block, unblock, remove_duplicates actions), Election Management (ElectionViewSet with start, stop, extend, results, publish_results, monitoring actions), Party Management (PartyViewSet), Candidate Management (CandidateViewSet), Voting (cast_vote, check_vote_status), Voter Portal (voter_profile, voter_elections, election_results_voter), Corrections (VoterCorrectionViewSet with approve, reject actions), and a Health Check endpoint.

`voting_backend/api/urls.py` — Registers five ViewSets with the DefaultRouter and defines nineteen manual URL paths for all standalone views and custom actions. The router's URLs are included at the root of the API namespace.

`voting_backend/api/email_service.py` — Implements five public email functions: `send_registration_confirmation_email`, `send_voter_approved_email`, `send_voter_rejected_email`, `send_correction_status_email`, and `send_new_election_email`. Each function constructs both HTML and plain-text email bodies, sends via `EmailMultiAlternatives`, and logs the outcome to the EmailNotification model via the

private `_log_email()` helper. The module uses Python's `logging` module for server-side error logging of failed sends.

`voting_backend/api/admin.py` — Registers all models with Django's built-in admin interface, providing a web-based database management panel useful during development. Custom list_display, list_filter, and search_fields are configured for each model to make the admin panel efficient for direct data management.

`voting_backend/api/signals.py` — Connects Django model signals (post_save) to trigger automated behaviors when model instances are saved. This provides an event-driven extension point for future automation without modifying view code.

`voting_backend/api/migrations/` — Contains five migration files documenting the evolution of the database schema from initial creation through the addition of father_name, VoterCorrection, and EmailNotification.

6.2 Frontend Source Files

`voting_app/lib/main.dart` — The application entry point. Sets up system UI overlay style (transparent status bar with dark icons), initializes the MultiProvider with AuthProvider, and defines the root MaterialApp. The SplashScreen stateful widget uses `AnimationController` with `SingleTickerProviderStateMixin` to drive elastic scale and opacity animations over 1500ms. After a 2-second delay, it calls `AuthProvider.init()` to restore persisted session and navigates to the appropriate screen with a 500ms fade transition.

`voting_app/lib/providers/auth_provider.dart` — Defines `AuthProvider extends ChangeNotifier`. Manages isLoggedIn, role, voterData, isLoading, and error state. Provides init(), adminLogin(), voterLogin(), and logout() methods. Persists token and role using SharedPreferences via the ApiService. Calls `notifyListeners()` after every state change to trigger UI rebuilds.

`voting_app/lib/services/api_service.dart` — Singleton HTTP client (553 lines). Groups methods by domain: Authentication (adminLogin, voterLogin, voterRegister, logout), Admin Dashboard (getDashboard), Voter Management (getVoters, createVoter, createVoterWithBytes, approveVoter, rejectVoter, blockVoter, unblockVoter, removeDuplicateVoters), Election Management (getElections, getElection, createElection, updateElection, startElection, stopElection, extendElection, getElectionResults, publishResults, getElectionMonitoring), Party Management (getParties, createParty, deleteParty), Candidate Management (getCandidates, createCandidate, updateCandidate, deleteCandidate), Voting (castVote, checkVoteStatus), Voter Portal (getVoterProfile, getVoterElections, getVoterElectionResults), and Corrections (submitCorrection, getCorrections, approveCorrection, rejectCorrection). The `_handleResponse()` private method centralizes response parsing, throwing `ApiException` for non-2xx status codes.

`voting_app/lib/utils/constants.dart` — Defines AppConstants class with baseUrl (`http://localhost:8000/api`), mediaBaseUrl (`http://localhost:8000`), appName (`Secure Voting`), appVersion (`1.0.0`), appTagline (`Your Vote, Your Voice`), and SharedPreferences storage keys (tokenKey, roleKey, voterIdKey, userDataKey).

`voting_app/lib/utils/theme.dart` — Defines `AppTheme` static class with color constants (primaryColor yellow, accentColor dark blue, successColor green, errorColor red, textPrimary, textSecondary, textLight, surfaceColor, dividerColor), gradient definitions (primaryGradient yellow, accentGradient blue, darkGradient dark, dangerGradient red), shadow lists (cardShadow, softShadow), and `lightTheme` ThemeData configuring InputDecoration, ElevatedButton, and overall Material3 theme using Inter font via GoogleFonts.

`voting_app/lib/screens/auth/login_screen.dart` — LoginScreen with mode toggle between Voter (Voter ID + Passcode) and Admin (Username + Password). Uses FadeTransition animation on mode switch. Delegates login to AuthProvider. Shows registration link for voter mode. Form validated using GlobalKey.

`voting_app/lib/screens/auth/register_screen.dart` — Multi-field voter registration form with image picker for photo upload (supporting both web XFile bytes and mobile File). Calls ApiService.voterRegister() with multipart upload.

`voting_app/lib/screens/admin/admin_dashboard_screen.dart` — Loads dashboard statistics from API on init and pull-to-refresh. Renders stats grid and navigation tiles to all management screens.

`voting_app/lib/screens/admin/voter_management_screen.dart` — Paginated voter list with status filter chips and search bar. Inline action buttons for approve/reject/block/unblock. Confirms actions with AlertDialogs.

`voting_app/lib/screens/admin/election_management_screen.dart` — Election list with lifecycle action buttons. Date/time pickers for election creation and extension. Navigation to monitoring and results screens.

`voting_app/lib/screens/admin/party_management_screen.dart` — Party CRUD with symbol image upload. Confirms deletion with dialog.

`voting_app/lib/screens/admin/candidate_management_screen.dart` — Candidate CRUD with election and party dropdowns loaded from API. Photo upload support.

`voting_app/lib/screens/admin/result_management_screen.dart` — Displays election results with ranked candidate list, winner highlight, vote counts, percentages, and participation rate. Publish Results button for closed elections.

`voting_app/lib/screens/admin/correction_management_screen.dart` — Lists all voter correction requests with status filter. Approve/Reject actions with notes input for rejection.

`voting_app/lib/screens/voter/voter_home_screen.dart` — Voter's main screen. Loads profile and elections on init. Displays profile card, voter card navigation tile, and election list. Each election card contextually shows Cast Vote (live, not voted), Vote Recorded (live, voted), View Results (closed, published), or Results Pending (closed, not published).

`voting_app/lib/screens/voter/vote_casting_screen.dart` — Candidate selection with animated selection state. Two-step progress indicator. Confirmation dialog with candidate summary. Success dialog with selectable SHA-256 transaction hash displayed in Roboto Mono font.

`voting_app/lib/screens/voter/voter_card_screen.dart` — Digital voter card rendering all registered details in a styled card layout.

`voting_app/lib/screens/voter/voter_results_screen.dart` — Voter-facing result screen showing ranked candidates, vote percentages displayed as progress bars, winner highlight, and total participation statistics.

---

7. Outputs

7.1 Splash Screen

When the application is launched, the splash screen appears on a pure white background. A yellow circular container with a vote icon inside scales up from 50% of its final size to full size using an elastic animation curve, simultaneously fading in from transparent to fully opaque over 1.5 seconds. Below the icon, the application name "Secure Voting" is displayed in bold black text, and the tagline "Your Vote, Your Voice" appears in dark grey. After 2 seconds, the app checks stored authentication state and transitions to the appropriate screen using a smooth fade-out and fade-in transition over 500 milliseconds.

7.2 Login Screen

The login screen presents a clean white interface with the app logo at the top. Below the logo is a segmented control allowing the user to toggle between Voter and Admin login modes. The active segment is highlighted with a gradient background — blue gradient for Voter mode, yellow gradient for Admin mode. The toggle switch animates smoothly between states using Flutter's AnimatedContainer.

In Voter mode, the form shows two fields: Voter ID with a badge icon, and Passcode with a lock icon and a visibility toggle. In Admin mode, the form shows Username with a person icon and Password with the same lock and visibility toggle. A gradient submit button fills the full width below the fields. In Voter

mode, a "Don't have an account? Apply for Voter" link appears below the button, navigating to the registration screen.

7.3 Registration Screen

The registration screen presents a scrollable form collecting all required voter information. Input fields are organized logically: personal details first (name, father's name, date of birth via a calendar picker, gender via dropdown), then contact details (address, email, mobile number), followed by a photo upload section showing either the selected image preview or a placeholder with a camera icon, and finally a passcode field. A full-width submit button at the bottom triggers the multipart form upload with a loading indicator while the request is in progress.

7.4 Admin Dashboard Screen

Upon admin login, the dashboard screen opens with a yellow header section displaying the application logo, the greeting "Admin Panel," and a logout icon button. The main content area, styled with a white/light surface color and rounded top corners, shows a 10-cell statistics grid in two columns. Each cell is a rounded card with an icon, a numerical count in large bold text, and a label. Cells use different icon colors to visually differentiate categories. Below the grid, five management navigation tiles lead to Voter Management, Election Management, Party Management, Candidate Management, and Result Management. A Corrections tile provides quick access to pending voter card correction requests.

7.5 Voter Management Screen

The voter management screen shows a tab-based filter row at the top (All, Pending, Approved, Rejected, Blocked) and a search bar. The list below shows voter cards with the voter's name in bold, their voter ID or mobile number below, the registration date, and a colored status badge (amber for pending, green for approved, red for rejected, dark grey for blocked). Tapping a voter card expands an action row. For pending voters, green "Approve" and red "Reject" buttons appear. For approved voters, a yellow "Block" button appears. For blocked voters, a blue "Unblock" button appears. Each action shows a loading state while the API request is in progress and displays a success snackbar upon completion.

7.6 Election Management Screen

The election management screen lists all elections as cards with a gradient status indicator, the election name, date range, candidate count, and total votes. A floating action button opens the Create Election dialog. Tapping an election reveals a bottom sheet with lifecycle action buttons contextually shown based on the election's current status. The monitoring view for a live election shows a live snapshot of vote counts per candidate in a ranked list with percentage bars.

7.7 Party Management Screen

The party management screen shows a grid of party cards, each displaying the party symbol image (or a flag placeholder if no symbol was uploaded), the party name, and a delete button. A "+" FAB opens the Create Party bottom sheet with name, description, and symbol upload fields. Deleting a party shows a confirmation dialog before proceeding.

7.8 Candidate Management Screen

The candidate management screen shows candidates grouped or listed with their name, photo (or a person placeholder), party name, election name, and current vote count. Admins can add candidates via a form that includes election and party dropdowns (loaded from the server), name, bio, and photo. Candidates can be edited or deleted with appropriate confirmation.

7.9 Results Screen (Admin)

The admin results screen for a closed election shows a ranked leaderboard of candidates with their vote counts, percentage bars representing their share of total votes, and a trophy icon next to the leading candidate. Below the leaderboard, summary statistics show total votes cast, total approved voters, and the participation rate as a percentage. A "Publish Results" button (highlighted with the primary yellow gradient) appears for elections whose results have not yet been published.

7.10 Voter Home Screen

The voter home screen opens with a yellow header featuring the app icon, "Welcome Back!" greeting text, and the voter's full name. The main content area shows a profile summary card with the voter's photo (or initial letter avatar), their full name, voter ID, and status badge. Below the profile card is a deep blue "My Voter Card" banner button that navigates to the voter card detail screen. Below that, the Elections section lists all live and closed elections.

Each election card shows the election name, a live/closed status indicator with a pulsing dot for live elections, a "VOTED" badge if the voter has already cast their vote in that election, and an action button area at the bottom. For a live election where the voter has not yet voted, a dark blue "Cast Your Vote" button appears. For a live election where the voter has already voted, a green "Vote Recorded" confirmation appears. For a closed election with published results, a yellow "View Results" button appears. For a closed election with unpublished results, a grey "Results Pending" indicator appears.

7.11 Vote Casting Screen

The vote casting screen opens with a dark gradient header and a two-step progress indicator. The candidate list below shows each candidate with their party symbol, name, and party name. When the voter taps a candidate, the card gains a highlighted border and a filled checkmark radio button with a

smooth 200ms animation. Once a candidate is selected, the "Submit Vote" button animates in at the bottom.

Tapping Submit Vote opens the confirmation dialog. The dialog shows the selected candidate's party symbol, name, and party in a summary card, with a red warning that the vote cannot be changed. The "Vote Now" button submits the vote. The success dialog that follows shows a large green checkmark, "Your vote has been cast securely," and below it, a grey container labeled "TRANSACTION HASH" displaying the 64-character SHA-256 hash in Roboto Mono font. The hash text is selectable, allowing the voter to long-press and copy it. A yellow "Back to Home" button closes the success flow.

7.12 Voter Card Screen

The voter card screen renders a digital facsimile of a government voter identity card. The card features the application branding at the top, the voter's photo on the left side in a bordered container, and personal details on the right: full name in bold, voter ID, father's name, date of birth, gender, address in smaller text, and a status badge showing APPROVED in green. The card is styled with rounded corners, a subtle shadow, and a color scheme that distinguishes it as an official document.

7.13 Voter Results Screen

The voter-facing results screen is accessible only when an election's results have been published by the admin. It shows the election name and status at the top, followed by a ranked list of candidates. Each entry shows the candidate's position number (with a gold medal icon for first place), their name, party name, vote count, and a proportional horizontal progress bar representing their vote percentage. A summary section at the bottom shows total votes cast and participation rate. If a winner has been determined (the candidate with the most votes when total votes > 0), their entry is highlighted with a distinct background.

7.14 Email Outputs

The system generates several distinct email outputs throughout its operation. The registration confirmation email has a subject line "📋 Voter Registration Received - Pending Approval" and informs the voter that their application is under review. The approval email has the subject "✅ Your Voter Application Has Been Approved!" and contains the voter's Voter ID, full personal details, and step-by-step login instructions, all formatted in a professional card layout. The rejection email has the subject "❌ Your Voter Application Status Update" and includes any rejection reason if provided. The election announcement email has the subject "🗳️ New Election: [Election Name]" and contains the election's details and a call-to-action to log in and vote. The correction status emails use "✅" or "❌" prefixes in the subject line to immediately communicate the outcome to the voter.

---

8. Future Enhancement

8.1 Blockchain-Based Vote Storage

The most impactful planned enhancement is the integration of blockchain technology for vote record storage. In the current system, votes are stored in a centralized relational database, which, while secure under normal operating conditions, represents a single point of trust. A blockchain-based approach would distribute vote records across a network of nodes, making retrospective manipulation of any vote computationally infeasible without controlling a majority of the network. Each vote's SHA-256 hash, already computed by the system, could serve directly as the transaction identifier on the blockchain. This would allow any voter to independently verify that their specific hash appears in the blockchain, confirming their vote was recorded. Public auditability of election results would then not require trusting any single organization's servers.

8.2 Server-Side Facial Recognition

The current biometric authentication delegates fingerprint verification entirely to the device's hardware security module. While this is privacy-preserving, it means the server cannot independently verify that the person using the device is the registered voter. A planned enhancement is to integrate server-side facial recognition during the login process. When a voter enables biometric login, their registered photo (already stored in the database) would be used as the facial recognition reference. At login, the voter would capture a selfie which the server would compare against the stored photo using a face recognition API. A match above a confidence threshold would authenticate the voter. This adds a second factor to biometric authentication that the server itself can verify, significantly strengthening identity assurance.

8.3 Real-Time Push Notifications

The current system relies entirely on email for voter communication. While email is reliable and universally accessible, it requires the voter to actively check their inbox. Integrating Firebase Cloud Messaging (FCM) for push notifications would enable real-time, in-app alerts. Voters would receive an immediate push notification when their registration is approved, when a new election goes live, when their correction request is processed, and when election results are published. The FCM device token for each voter's device would be stored alongside the biometric token in the voter's profile, and the notification service would call the FCM REST API in addition to sending emails.

8.4 Multi-Language Support

India and many other democracies have large voter populations whose primary language is not English. A future version of the system will incorporate Flutter's `intl` package for internationalization, with

translation files (ARB format) for all UI strings. The backend will support locale-aware email generation, sending approval and notification emails in the voter's preferred language as specified during registration. This significantly lowers the barrier to participation for voters with limited English literacy.

8.5 Offline Voting Capability

In areas with unreliable internet connectivity, the inability to cast a vote due to a network error could disenfranchise voters. A future version will explore offline vote queuing: the voter selects their candidate and confirms their intent while offline, and the vote is encrypted and stored locally. When the device regains connectivity within the election's active window, the queued vote is automatically submitted. This requires careful cryptographic design to ensure that locally stored votes cannot be tampered with between the time of casting and the time of submission.

8.6 Web-Based Admin Portal

While the Flutter app can run on web browsers through Flutter Web compilation, a dedicated server-rendered web admin portal would provide a superior experience for election officials working at desktop workstations. The portal would offer advanced features such as bulk voter import from CSV files, comprehensive email notification history browsing with filtering and re-send capability, election analytics dashboards with interactive charts (voter turnout over time, geographic distribution of voters if location data is captured), and detailed audit log viewing. The Django backend is already fully capable of serving such a web frontend.

8.7 Two-Factor Authentication for Admins

The current admin login uses only username and password. Given that the admin account has complete control over elections and voter data, adding two-factor authentication (2FA) is an important security enhancement. This could be implemented using time-based one-time passwords (TOTP) — the admin scans a QR code with an authenticator app during initial setup, and subsequently must enter a 6-digit time-sensitive code from the app in addition to their password. The `django-two-factor-auth` package provides a ready-made implementation that integrates with Django's authentication system.

8.8 Advanced Audit Logging

A comprehensive system audit log that records every significant administrative action — voter status changes, election lifecycle transitions, result publications, correction approvals — with timestamps and the admin's username would greatly enhance accountability. This log, read-only and append-only, would serve as a complete forensic record of all actions taken during and around each election, providing verifiable evidence in case of any dispute about the conduct of the election.

8.9 Voter Turnout Analytics

Post-election analytics that visualize voter turnout patterns — comparing participation rates across elections, identifying time-of-day peaks in voting activity, showing which voter demographics (age, gender) participated at higher rates — would be valuable for election planners in future cycles. These analytics could be displayed in the admin dashboard using Flutter charts or in the web portal using JavaScript charting libraries, driven by aggregate database queries from the backend.

---

9. Conclusion

The Secure Mobile Biometric Voting System represents a complete and functional implementation of a mobile-first digital voting platform, successfully achieving all of its stated objectives. Through the combination of Flutter's cross-platform mobile development framework and Django REST Framework's powerful backend capabilities, the project delivers a system that is simultaneously user-friendly for voters, powerful for administrators, and technically secure at every layer.

The voter experience has been designed to feel as natural and trustworthy as possible. The registration process guides users through each required field with clear labels and inline validation. The voter card provides a familiar, document-like representation of their identity. The vote casting interface is deliberate in its design — showing clear candidate information, requiring an explicit confirmation step, and providing a cryptographic receipt that voters can retain. These design decisions collectively reduce the psychological barrier to adopting digital voting.

The administrator experience is equally well-considered. The dashboard gives election officials an at-a-glance overview of the system state, and every management operation — from approving a voter to publishing election results — is accessible through an intuitive interface without requiring any technical knowledge of the underlying systems. Automated email notifications eliminate the most time-consuming manual communication tasks that election officials would otherwise face.

From a technical standpoint, the system implements security best practices at multiple levels. Passcodes are hashed using PBKDF2 with a random salt before storage, making them irrecoverable even if the database is accessed without authorization. Token-based authentication invalidates sessions server-side at logout, preventing token reuse. The one-vote-per-election constraint is enforced both at the application logic level and at the database level through unique constraints, providing redundant protection against double voting. The SHA-256 vote hash provides cryptographic proof of participation that neither the voter nor the administrator can fabricate or deny.

The email notification system transforms what could be a cold, bureaucratic process into a communicative, transparent one. Voters are kept informed at every step through well-designed HTML emails that maintain the application's branding and tone. Every email is logged in the database, creating

an auditable trail of all communications. This transparency builds trust — a critical factor in any electoral system.

Looking forward, the enhancements described in the Future Enhancement section chart a clear path toward an even more secure, scalable, and feature-complete system. Blockchain integration would move the system from centralized trust to distributed verification. Server-side facial recognition would close the remaining gap in biometric identity assurance. Push notifications would make the voter experience more immediate and engaging. Multi-language support would extend democratic participation to a broader population.

In conclusion, this project demonstrates not only the technical feasibility of mobile-based digital voting but also the importance of combining security engineering with thoughtful user experience design. A secure voting system that users find confusing or intimidating will not achieve its democratic goals, no matter how cryptographically sound it is. The Secure Mobile Biometric Voting System succeeds in making security and usability work together, proving that digital democracy does not require a trade-off between the two.

---

10. Bibliography

1. Django Software Foundation. (2024). *Django Documentation — Version 4.x*. Retrieved from https://docs.djangoproject.com/
2. Django REST Framework. (2024). *Django REST Framework Documentation*. Retrieved from https://www.django-rest-framework.org/
3. Flutter Team, Google. (2024). *Flutter Developer Documentation*. Retrieved from https://flutter.dev/docs
4. Dart Team, Google. (2024). *Dart Programming Language Documentation*. Retrieved from https://dart.dev/guides
5. Flutter Provider Package. (2024). *Provider — State Management for Flutter*. Retrieved from https://pub.dev/packages/provider
6. Google Fonts Flutter Package. (2024). *google_fonts — Flutter Package*. Retrieved from https://pub.dev/packages/google_fonts
7. Shared Preferences Flutter Package. (2024). *shared_preferences — Flutter Package*. Retrieved from https://pub.dev/packages/shared_preferences
8. Alvarez, R. M., & Hall, T. E. (2008). *Electronic Elections: The Perils and Promises of Digital Democracy*. Princeton University Press.

9. Schneier, B. (2015). *Data and Goliath: The Hidden Battles to Collect Your Data and Control Your World*. W. W. Norton & Company.

10. Kohno, T., Stubblefield, A., Rubin, A. D., & Wallach, D. S. (2004). *Analysis of an Electronic Voting System*. IEEE Symposium on Security and Privacy, 2004.

11. Clarkson, M. R., Chong, S., & Myers, A. C. (2008). *Civitas: Toward a Secure Voting System*. IEEE Symposium on Security and Privacy, 2008.

12. Adida, B. (2008). *Helios: Web-based Open-Audit Voting*. Proceedings of the 17th USENIX Security Symposium.

13. Acemoglu, D., & Robinson, J. A. (2012). *Why Nations Fail: The Origins of Power, Prosperity and Poverty*. Crown Publishers.

14. OWASP Foundation. (2024). *OWASP Mobile Security Testing Guide (MSTG)*. Retrieved from https://owasp.org/www-project-mobile-security-testing-guide/

15. Python Software Foundation. (2024). *Python 3 Standard Library Documentation — hashlib*. Retrieved from https://docs.python.org/3/library/hashlib.html

16. National Institute of Standards and Technology. (2012). *FIPS PUB 180-4: Secure Hash Standard (SHS)*. U.S. Department of Commerce.

17. RFC 6238. (2011). *TOTP: Time-Based One-Time Password Algorithm*. Internet Engineering Task Force (IETF).

18. Buterin, V. (2014). *Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform*. Ethereum Foundation.

19. Flutter Architecture Samples. (2024). *Various Flutter State Management Patterns*. Retrieved from https://fluttersamples.com/

20. Python Cryptographic Authority. (2024). *Cryptography — Python Cryptography Package Documentation*. Retrieved from https://cryptography.io/

This project report documents the complete design, implementation, and analysis of the Secure Mobile Biometric Voting System. All source code is original and developed for academic purposes.

System Version: 1.0.0 | Report Date: 2026